

Progress Report — SEBI ICDR Rule Extraction: v5 → v6 → v7

Date: May 9, 2026

1. Overview

This report covers the full progression of the SEBI ICDR rule extraction pipeline across three versions (v5, v6, v7), all code changes made in the v7 sprint, and a diagnosis of what worked, what regressed, and what to fix next.

v5 and v6 scores are from the official scoring runs against the 143-rule gold standard ([gold_standard_regs_4_23.jsonl](#)). v7 was scored after applying all v7 code changes; the v7 run predates the final round of bug fixes (carryover ordering, splitter guard), so some improvements will appear only in v8.

2. Three-Version Headline Comparison

Metric	v5	v6	v7	v5→v6	v6→v7
Gold total	143	143	143	—	—
Extracted (unique IDs)	118	103	113	−15	+10
True Positives (TP)	92	100	88	+8	−12
False Negatives (FN)	51	43	55	−8	+12
False Positives (FP)	26	3	25	−23	+22
Recall	64.3%	69.9%	61.5%	+5.6 pp	−8.4 pp
Precision	78.0%	97.1%	77.9%	+19.1 pp	−19.2 pp
F1	70.5%	81.3%	68.75%	+10.8 pp	−12.6 pp
Duplicate rule_ids	1	1	1	0	0

Key takeaway: v6 was a strong improvement over v5 (+10.8 pp F1) largely by reducing False Positives from 26 to 3. v7 reversed that precision gain, introducing 22 new FPs via the over-eager list-item splitter and carryover-hint changes — even though the underlying recall improvements for Regs 10, 11, 12 are real and carry forward.

3. Per-Regulation Breakdown — All Three Versions

Reg	Gold	v5 TP	v5 FP	v6 TP	v6 FP	v7 TP	v7 FP	Trend
4	1	1	0	1	0	1	0	Stable ✓

Reg	Gold	v5 TP	v5 FP	v6 TP	v6 FP	v7 TP	v7 FP	Trend
5	9	5	1	6	0	5	2	v6 best; v7 slight regression
6	23	18	2	11	0	12	1	v5 best recall; v6 tighter anchoring hurt
7	13	9	2	10	0	10	5	v6 TP gain held; v7 adds FPs
8	10	3	2	3	0	1	0	Persistent gap; v7 regression
8A	4	0	0	3	0	3	0	v6/v7 improvement over v5
9	2	2	0	2	0	2	0	Perfect all versions
10	10	10	0	9	1	10	0	v7 fix recovered v5 level ✓
11	5	4	0	4	0	5	0	v7 achieves full coverage ✓
12	2	2	0	1	0	2	0	v7 fix recovered missing rule ✓
13	5	1	1	5	0	3	5	v6 was perfect; v7 severe regression
14	13	11	3	9	0	7	0	v5 best; deep provisos persist
15	14	4	8	10	0	5	3	v6 big gain; v7 regression
16	6	1	4	5	0	3	3	v6 big gain; v7 regression
17	10	7	3	5	0	5	6	v5 best recall; v7 adds FPs
18	2	2	0	2	0	2	0	Perfect all versions
19	1	1	0	1	0	1	0	Perfect all versions
20	1	1	0	1	0	1	0	Perfect all versions
21	4	2	0	4	0	4	0	v6/v7 achieve full coverage ✓
22	2	2	0	2	0	1	0	v7 regression (proviso dropped)
23	6	6	0	6	0	5	0	v7 regression (proviso dropped)

Notable version-over-version observations

v5 → v6 wins: Reg 8A (0→3 TP), Reg 13 (1→5 TP), Reg 15 (4→10 TP), Reg 16 (1→5 TP), Reg 21 (2→4 TP). Achieved by tighter anchoring and better two-pass prompting, which cut FPs from 26 to 3.

v5 → v6 losses: Reg 6 dropped from 18→11 TP — v6's stricter anchoring wrongly rejected continuation-page sub-clauses of 6(3) that v5 had captured. This is the root cause that triggered the carryover-hint improvements in v7.

v6 → v7 confirmed wins: Reg 10 (9→10), Reg 11 (4→5), Reg 12 (1→2). These are real, durable improvements from targeted ID renames and prompt fixes.

v6 → v7 regressions: Regs 5, 13, 15, 16, 17 all gained FPs. The `split_merged_lettered_items()` function was too aggressive and misfired on regulation structures where the LLM hadn't actually merged items. Regs 22 and 23 lost provisos, likely from carryover-hint depth changes affecting window boundaries.

4. All Code Changes Made in the v7 Sprint

All changes committed as [71b5326](#) to github.com/saunakp123/compliance.

4.1 `scripts/rule_extraction/regulation_identifier.py`

Fix 1 — `HEADING_NUMBER_RE` activated in `detect_allowed_regs()`

The regex was compiled at module level but never called. Added a third `finder` loop to catch bare heading numbers (4., 22.) that regulations with no sub-clauses produce. Guard: only accept $1 \leq n \leq 100$.

Fix 2 — `REG_HEADER_RE` lowercase tolerance

Changed final character class `[A-Z]` → `[A-Za-z]` so headings like 4. applicability match and enter `pre_identify_regulations()`'s found set.

Fix 3 — `FOOTNOTE_TAIL_RE` Pattern C

Added `FOOTNOTE_TAIL_RE` to catch cross-page amendment footnote continuation fragments ((Amendment) Regulations...) that survived Pattern A/B stripping. Applied in `strip_footnotes_with_linkage()` before the blank-line collapse.

Fix 4c — `reg_context` carryover-hint reinforcement

When both `visible_regs` and `carryover_hint` are active in `identify_regulations()`, a NOTE paragraph is appended telling the model the continuation hint takes priority over the top-level list for items at the window start.

Fix 5a — List-item separation block in `_PASS1_SYSTEM`

Added `LISTS OF LETTERED OR ROMAN-NUMERAL ITEMS` instruction: each lettered/roman item is a separate output object. Includes a four-object example.

Fix 5b — Two new bullets in `_PASS1_USER_TEMPLATE`

- One object per lettered/roman item, never merge two.
- Parent `clause_text` ends at the colon/dash before a list.

Fix 5c — `split_merged_lettered_items()` deterministic splitter

New function before `identify_regulations()`. Detects clause objects whose `reg_number` ends in (a)–(e) and whose `clause_text` contains sibling markers (b. ..., c. ...), then splits into one object per item. Called at the end of `identify_regulations()`. **Note:** this function fires too broadly in v7 — see Priority 1 fix in Section 6.

Pass 1 / Pass 2 prompt additions

- **AMENDMENT-INSERTED SUB-REGULATIONS** block in `_PASS1_SYSTEM`.
 - **EXPLANATION BLOCKS** guidance with plain wording (no `{parent_reg}` placeholder).
 - `clause_text` cap raised from 300/500 → **1200 chars** in both `_PASS1_USER_TEMPLATE` and `build_targeted_extraction_prompt()`.
 - **NESTED PROVISOS** and **MAIN RULE + PROVISIO SEPARATION** instructions in the Pass 2 prompt builder.
-

4.2 `scripts/llm_extract_rules.py`

Fix 4a — Strip terminal letter from `prev_window_last_subclause`

Before carrying a Pass 1 clause forward as the next window's context, strip any trailing `(a)–(e)` suffix: `"6(3)(iv)(a)"` → `"6(3)(iv)"`. This ensures the hint points at the structural parent (level for roman-numeral siblings) not a terminal leaf.

Fix 4b — Rewritten carryover hint

Two-depth hint replacing the old single-level one:

- Single-letter items `(a., b.)` → parent = `prev_window_last_subclause`
- Roman-numeral / integer items `(i., v., (3))` → parent = `parent_of(...)`
- Provisos → `prev_window_last_subclause`
- Explicit **NEVER** lines and concrete `reg_number` examples.

`expand_detected_regs()` — alphanumeric base expansion

`detect_allowed_regs()` returns integers only, so `"8A"` in structural strings never caused `8` to enter `allowed_regs`. The new function takes `visible_reg_strings` from `pre_identify_regulations()` and adds the numeric base for any alphanumeric entry (`"8A"` → add `8`).

KNOWN_ID_RENAMES with **normalize follow-through**

`{"ICDR_10_iv": "ICDR_10_1_d_iv"}` added. After renaming, `normalize_rule_identifier()` is called to keep `rule_id_norm`, `lean_id`, `sub_id` consistent.

SUSPICIOUS_IDS flagging

`ICDR_15_1_b_proviso` and `ICDR_16_1_a_proviso` are marked `status: needs_review` rather than silently accepted.

--eval-gold **needs_review** logic consolidated

Two separate flagging branches merged into one condition.

4.3 `data/schema/icdr_structure.json`

Corrected a duplicate "number": "5" entry that appeared under Chapter XII instead of Chapter II. All Regulations 4–23 now correctly mapped to INITIAL PUBLIC OFFER ON MAIN BOARD.

5. What Worked vs. What Regressed

Confirmed improvements ☒

Fix	Regulation	Effect
KNOWN_ID_RENAMES	Reg 10	9→10 TP; full coverage
Pass 2 prompt (NESTED PROVISOS)	Reg 11	4→5 TP; full coverage
Main+proviso separation prompt	Reg 12	1→2 TP; full coverage
HEADING_NUMBER_RE fallback	Reg 4	Maintained 1/1 TP
Reg 8A anchoring	Reg 8A	Maintained 3/4 TP from v6

Regressions and root causes ☐

Regulation	v6 TP	v7 TP	v7 FP	Root Cause
Reg 13	5	3	5	split_merged_lettered_items() split 13_c into 13_c_a/13_c_b — wrong; those aren't lettered sub-items in the gold hierarchy
Reg 15	10	5	3	Splitter misfired on 15_1_a_iv, created 15_1_a_c (nonsensical ID)
Reg 16	5	3	3	Carryover hint depth change: 16_1 and 16_explanation emitted instead of 16_1_proviso
Reg 17	5	5	6	Model adds extra (1) level: emits 17_1_a/17_1_b but gold expects 17_a/17_b
Reg 8	3	1	0	allowed_regs was computed before visible_regs_with_prev was defined (ordering bug fixed in latest commit but post-v7-run)
Reg 14	9	7	0	Deep provisos (14_proviso, 14_proviso_2) still beyond 1200-char reach of parent clause body
Reg 22, 23	2, 6	1, 5	0	Window-boundary provisos dropped — carryover depth change may have affected adjacent windows

6. Recommended Next Fixes (v8)

Priority 1 — Constrain split_merged_lettered_items() (High Impact, Low Risk)

Add a guard: only split when the first chunk ends with a list-introducing colon or dash.

```
# In split_merged_lettered_items(), after computing first_text:
if not re.search(r"[::-]\s*$", first_text):
    result.append(item)
    continue
```

This will eliminate the Reg 13 and 15 FPs without losing any correct splits.

Priority 2 — Fix Reg 17 hierarchy with renames (Medium Impact, Trivial Effort)

Gold expects `ICDR_17_a`, `ICDR_17_b`, `ICDR_17_c` (direct children of Reg 17). Model emits `ICDR_17_1_a`, `ICDR_17_1_b` (extra (1) level). Add to `KNOWN_ID_RENAMES`:

```
"ICDR_17_1_a": "ICDR_17_a",
"ICDR_17_1_b": "ICDR_17_b",
"ICDR_17_1_a_proviso": "ICDR_17_a_proviso",
"ICDR_17_1_b_proviso": "ICDR_17_b_proviso",
```

Priority 3 — Verify Reg 8 anchoring fix (Medium Impact, Already Coded)

The `allowed_regs` ordering bug (computing it before `visible_regs_with_prev` was defined) was fixed in the latest commit but after the v7 run. The next extraction run should automatically verify whether `8A` → `allow 8` now works and `ICDR_8`, `ICDR_8_proviso`, `ICDR_8_proviso_2` are no longer dropped by anchoring.

Priority 4 — Deep provisos for Reg 14 (Medium Impact, Medium Effort)

`ICDR_14_proviso` and `ICDR_14_proviso_2` sit after long parent body text that exhausts the 1200-char cap. Add a targeted second Pass 2 call when the parent `clause_text` is truncated (doesn't end with sentence-final punctuation):

```
if not clause_text.rstrip().endswith((".", ";")):
    # Call Pass 2 again with "extract ONLY the proviso / Provided that clauses"
    # using a larger text window
```

Priority 5 — Reg 6(3) continuation pages (High Impact, High Effort)

`ICDR_6_3_v` through `ICDR_6_3_ix` remain FN. The carryover hint is now more precisely structured (Fix 4a/4b), but the v7 run used the old hint. Re-run with the latest code to measure the actual gain before adding further logic.

7. Score Report Files

Version	File	Score report
v5	data/processed/rules_refactored_v5.jsonl	data/reports/v5_score_report.json

Version	File	Score report
v6	data/processed/rules_refactored_v6.jsonl	(included as baseline in v7 report)
v7	data/processed/rules_refactored_v7.jsonl	data/reports/v7_score_report.json

8. Next Extraction Run Command (v8)

```
# Step 1 – Extract
py scripts/llm_extract_rules.py `
  --pdf data/input/ICDR_rules_4_22.pdf `
  --out data/processed/rules_refactored_v8.jsonl `
  --model qwen2.5:14b-instruct `
  --window 2 --overlap 1 --reg-filter 4 23 `
  --timeout 600 --debug `
  --debug-dir data/processed/debug_refactored_v8

# Step 2 – Score
py scripts/score_extraction.py `
  --extracted data/processed/rules_refactored_v8.jsonl `
  --gold data/gold_standard/gold_standard_regs_4_23.jsonl `
  --baseline data/processed/rules_refactored_v6.jsonl `
  --output data/reports/v8_score_report.json
```

Target: F1 > 81.3% (recover v6 level) with Priority 1–2 fixes, then push toward 90% with Priority 3–5.

Generated: May 9, 2026. Pipeline v0.3.0. Extraction model: qwen2.5:14b-instruct. v5 score run: `py scripts/score_extraction.py --extracted data/processed/rules_refactored_v5.jsonl --gold ...` v7 score run: `py scripts/score_extraction.py --extracted data/processed/rules_refactored_v7.jsonl --baseline data/processed/rules_refactored_v6.jsonl ...`